

RAPPORT DE VEILLE

A. Introduction et méthodologie de veille

Ce rapport s'inscrit dans le cadre d'une mission de conseil technique pour SignalWatch, une startup IoT dont la plateforme de monitoring de capteurs industriels traite 10 000 événements par minute. Face à un choix architectural structurant, à savoir celui du protocole de communication entre microservices, le CTO a souhaité dépasser le comparatif théorique pour obtenir des mesures concrètes et reproductibles.

La démarche adoptée est une veille technico-pratique menée sur une semaine, combinant recherche documentaire et implémentation simultanée. Plutôt que de séparer une phase de lecture et une phase de développement, les deux ont été conduites en parallèle : chaque concept étudié était immédiatement mis en pratique, ce qui a permis de valider la compréhension et d'ancrer les résultats dans un contexte réel.

Sources consultées

La veille s'est appuyée sur des sources primaires et institutionnelles afin de garantir la fiabilité des informations :

- **Documentation officielle gRPC** : grpc.io et grpc.io/blog
- **Documentation Protocol Buffers** : protobuf.dev
- **Google Cloud Blog** : article de lancement gRPC 1.0 et articles d'architecture microservices
- **Google Developers Blog** : article de lancement open source gRPC (2015)
- **Documentation officielle Gin** : gin-gonic.com
- **Documentation k6** : k6.io/docs
- **Documentation ghz** : ghz.sh/docs
- **CNIL** : cnil.fr pour les obligations RGPD
- **RGESN** : référentiel général d'écoconception des services numériques

Critères de fiabilité

Pour valider la pertinence d'une source, trois critères ont été appliqués :

1. L'origine de la source (documentation officielle ou auteur identifié avec une expertise reconnue).
2. La date de publication (privilégier les sources postérieures à 2020 pour les aspects techniques).
3. La cohérence avec les résultats obtenus en pratique.

Répartition du travail

Ce projet a été conduit individuellement. La veille a couvert l'ensemble des thématiques du sujet : protocoles de communication (REST, gRPC, GraphQL, message brokers), outils de benchmark (k6, ghz), écoconception numérique et réglementation RGPD appliquée à l'IoT industriel.

B. Panorama des protocoles de communication

a. REST/HTTP : Representational State Transfer

REST est un style architectural défini par Roy Fielding dans sa thèse de doctorat en 2000. Il repose sur le protocole HTTP/1.1 et des conventions sémantiques standardisées. C'est aujourd'hui le protocole de communication le plus répandu pour les APIs web publiques et les architectures microservices orientées client.

Cas d'usage REST est particulièrement adapté aux APIs publiques consommées par des clients variés (navigateurs, applications mobiles, partenaires externes), aux interfaces CRUD simples, et aux systèmes où l'interopérabilité et la facilité d'intégration priment sur la performance brute. C'est le choix naturel lorsque les consommateurs de l'API ne maîtrisent pas d'outils spécifiques.

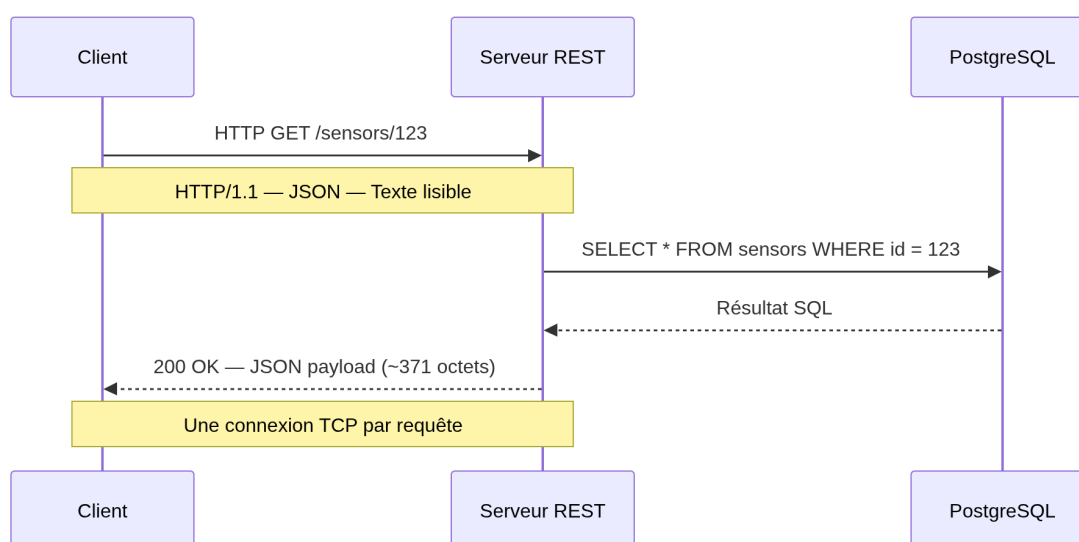
Forces

- **Universellement supporté** : tout client HTTP peut consommer une API REST sans bibliothèque spécifique.
- **Lisible et débogable** : les requêtes et réponses JSON sont lisibles par un humain.
- **Écosystème mature** : outils de test (Postman, curl), documentation (Swagger/OpenAPI), proxies et passerelles API.
- **Sans état (stateless)** : chaque requête est indépendante, ce qui facilite la scalabilité horizontale.
- **Mise en cache native** : les mécanismes HTTP (ETags, Cache-Control) sont directement exploitables.

Faiblesses

- **Verbosité du format JSON** : les payloads texte sont significativement plus lourds que les formats binaires.
- **HTTP/1.1 par défaut** : une connexion TCP par requête, pas de multiplexage natif.
- **Pas de contrat strict** : l'absence de schéma imposé peut entraîner des incompatibilités entre versions.
- **Over-fetching et under-fetching** : le client reçoit souvent plus ou moins de données que nécessaire.
- **Pas adapté au streaming** : REST n'est pas conçu pour les flux de données continus.

Schéma de fonctionnement :



b. gRPC/HTTP2 : Google Remote Procedure Call

gRPC est un framework RPC open source développé par Google et publié en 2015. Il repose sur HTTP/2 et utilise Protocol Buffers (Protobuf) comme format de sérialisation binaire. Conçu initialement pour les communications internes à Google (système Stubby, qui traitait plus de 10 milliards de requêtes par seconde), gRPC est aujourd'hui maintenu par la CNCF (Cloud Native Computing Foundation) et adopté par des entreprises comme Netflix, Cloudflare et Cisco.

Cas d'usage gRPC est particulièrement adapté aux communications inter-services dans une architecture microservices, aux systèmes à fort volume de requêtes où la latence est critique, aux flux de données continus (streaming), et aux environnements polyglottes où plusieurs langages cohabitent. C'est le protocole de référence pour les communications internes entre services backend.

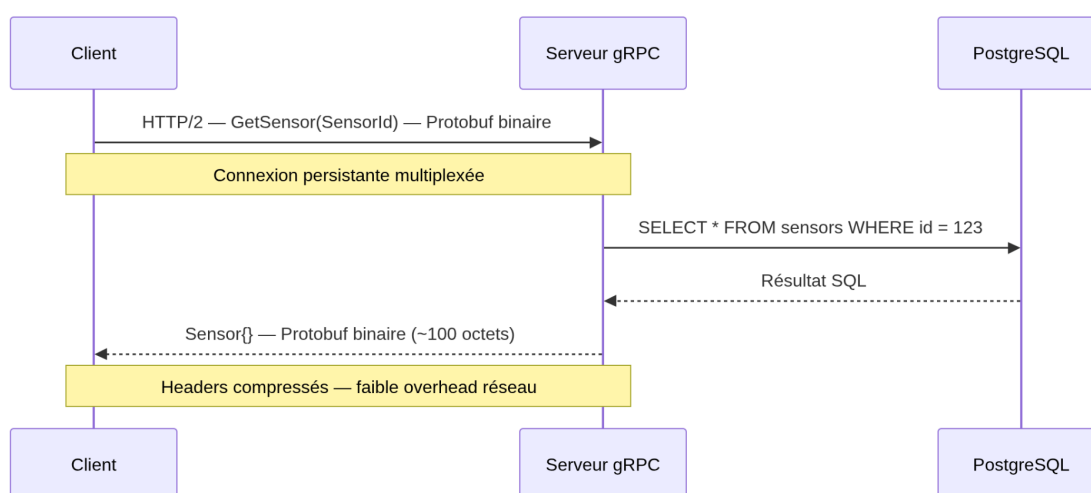
Forces

- **Performance** : sérialisation Protobuf binaire 3 à 5 fois plus compacte que JSON, HTTP/2 avec multiplexage des connexions.
- **Contrat strict** : le fichier `.proto` impose un schéma partagé entre client et serveur, évitant les incompatibilités.
- **Streaming natif** : supporte les flux unidirectionnels et bidirectionnels sans configuration supplémentaire.
- **Génération de code** : les clients et serveurs sont générés automatiquement depuis le `.proto` dans tous les langages supportés.
- **Faible latence** : multiplexage HTTP/2, compression des headers, connexions persistantes.

Faiblesses

- **Lisibilité réduite** : le format binaire Protobuf n'est pas lisible par un humain, ce qui complique le débogage.
- **Outils moins universel** : nécessite des outils spécifiques (grpcurl, Postman gRPC) contrairement à curl.
- **Support navigateur limité** : gRPC-Web est nécessaire pour les clients web, avec des limitations.
- **Courbe d'apprentissage** : la définition du `.proto` et la génération de code ajoutent une étape au workflow.
- **Moins adapté aux APIs publiques** : l'écosystème REST reste dominant pour les APIs consommées par des tiers.

Schéma de fonctionnement :



c. GraphQL

GraphQL est un langage de requête pour APIs développé par Facebook en 2012 et open sourcé en 2015. Contrairement à REST qui expose des endpoints fixes, GraphQL expose un endpoint unique via lequel le client spécifie exactement les données qu'il souhaite recevoir.

Cas d'usage GraphQL est particulièrement adapté aux applications avec des interfaces complexes et des besoins de données variables selon les vues (applications mobiles, dashboards), aux APIs consommées par plusieurs types de clients aux besoins différents, et aux systèmes où la flexibilité des requêtes est prioritaire.

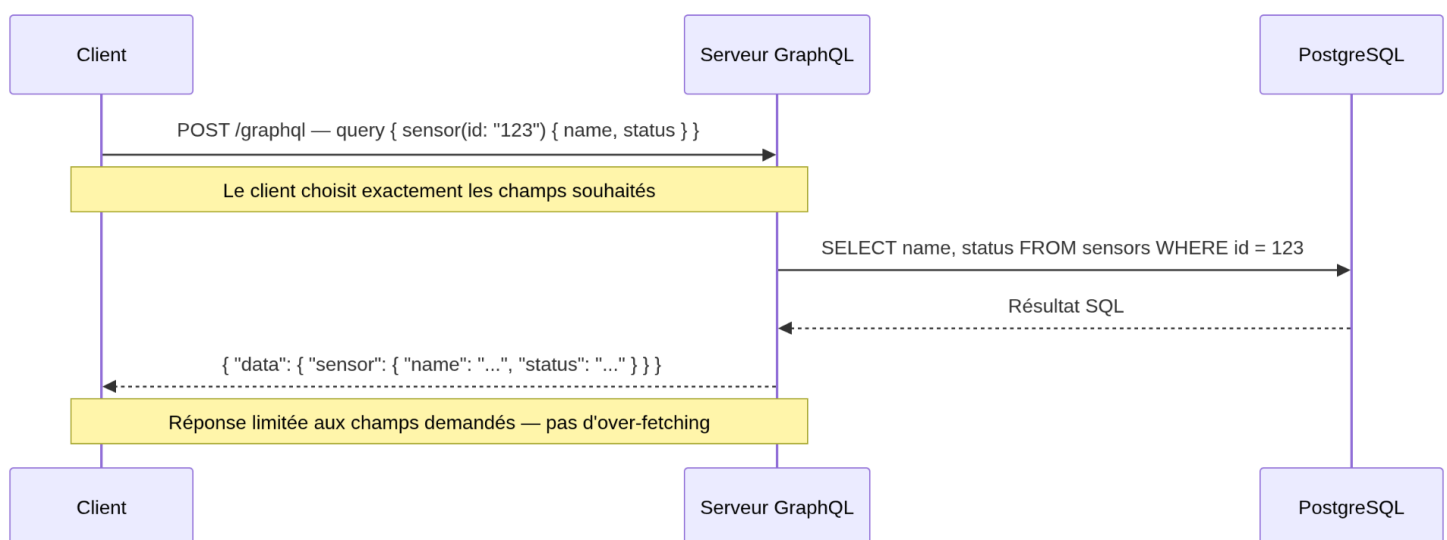
Forces

- **Pas d'over-fetching ni d'under-fetching** : le client demande exactement ce dont il a besoin.
- **Endpoint unique** : simplifie la gestion des versions et la documentation.
- **Introspection native** : le schéma est auto-documenté et interrogeable.
- **Agrégation de sources** : un seul appel GraphQL peut agréger des données de plusieurs services.
- **Typage fort** : le schéma GraphQL impose un contrat strict entre client et serveur.

Faiblesses

- **Complexité serveur** : le resolver pattern et la gestion du problème N+1 nécessitent une expertise spécifique.
- **Mise en cache difficile** : l'endpoint unique et les requêtes dynamiques contournent les mécanismes de cache HTTP.
- **Performances imprévisibles** : une requête mal formulée peut générer une charge serveur excessive.
- **Moins adapté aux flux temps réel** : les subscriptions GraphQL ajoutent de la complexité.
- **Overhead pour les cas simples** : REST reste plus simple pour des APIs CRUD basiques.

Schéma de fonctionnement :



d. Message brokers : Kafka et RabbitMQ

Les message brokers sont des middlewares de communication asynchrone. Contrairement à REST et gRPC qui fonctionnent sur un modèle requête/réponse synchrone, les brokers reposent sur un modèle publish/subscribe où les producteurs et consommateurs de données sont découplés et n'ont pas besoin de se connaître mutuellement.

Kafka

Apache Kafka est une plateforme de streaming distribué créée par LinkedIn en 2011 et open source la même année. Elle est conçue pour traiter des volumes massifs de données en temps réel avec une haute durabilité.

Cas d'usage Kafka est particulièrement adapté à l'ingestion de données à très haut volume (IoT, logs, événements applicatifs), aux architectures event-driven, et aux pipelines de données entre microservices. C'est le choix de référence pour les systèmes qui doivent traiter des millions d'événements par seconde avec une garantie de livraison.

Forces

- **Throughput exceptionnel** : des millions de messages par seconde.
- **Durabilité** : les messages sont persistés sur disque et rejouables.
- **Scalabilité horizontale** : partitionnement natif des topics.
- **Tolérance aux pannes** : réplication des données entre brokers.
- **Rétention configurable** : les messages peuvent être conservés des jours ou des semaines.

Faiblesses

- **Complexité opérationnelle** : nécessite ZooKeeper ou KRaft, difficile à opérer en production.
- **Latence plus élevée qu'un appel gRPC direct** : pas adapté aux échanges request/response.
- **Courbe d'apprentissage importante** : concepts de topics, partitions, offsets, consumer groups.
- **Surdimensionné pour les petits volumes** : overhead infrastructure significatif.

RabbitMQ

RabbitMQ est un message broker open source créé en 2007, basé sur le protocole AMQP. Plus simple à opérer que Kafka, il est orienté routage de messages plutôt que streaming massif.

Cas d'usage RabbitMQ est adapté aux files de tâches asynchrones, aux systèmes de notification, et aux architectures microservices nécessitant un routage flexible des messages. Il est privilégié quand la simplicité opérationnelle prime sur le volume.

Forces

- **Routage flexible** : exchanges, queues et bindings permettent des topologies complexes.
- **Protocoles multiples** : AMQP, MQTT, STOMP.

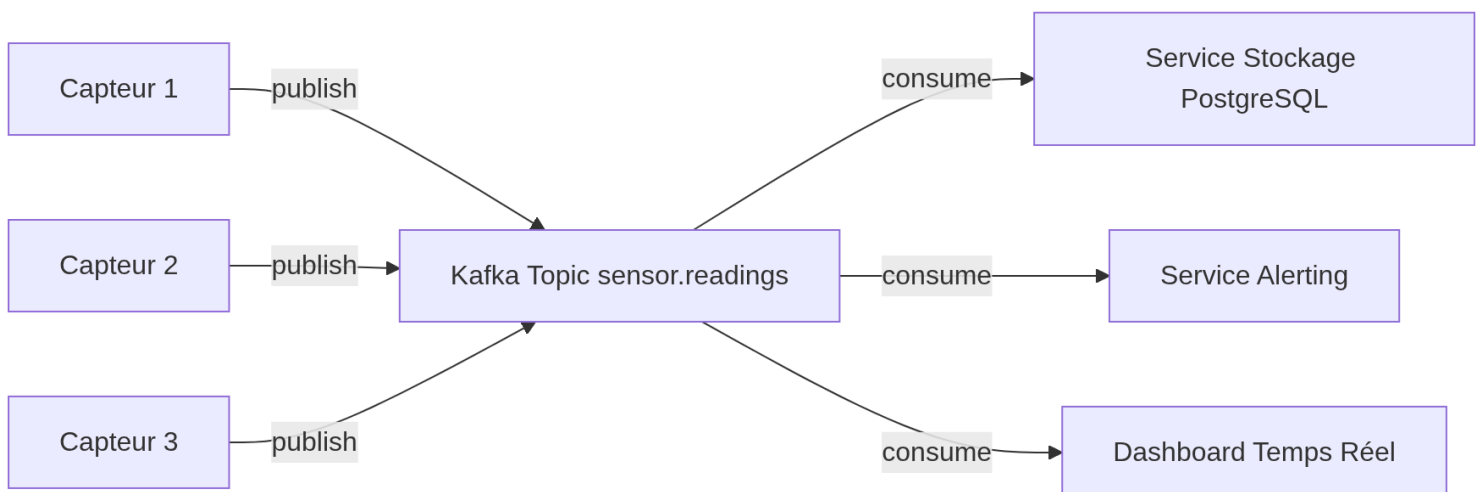
- **Interface de gestion** : dashboard web intégré pour monitorer les queues.
- **Simplicité** : plus simple à opérer que Kafka pour des volumes modérés.
- **Acquittement des messages** : garantie de livraison configurable.

Faiblesses

- **Pas conçu pour le streaming massif** : performances limitées face à Kafka à très haut volume.
- **Persistance moins robuste que Kafka** : les messages ne sont pas rejouables par défaut.
- **Scalabilité plus limitée** : clustering plus complexe à mettre en œuvre.

Pertinence pour SignalWatch Dans le contexte SignalWatch (10 000 événements/minute), un broker comme Kafka serait complémentaire à gRPC, pas concurrent. Les capteurs publieraient leurs lectures dans un topic Kafka, et plusieurs services consommeraient ces données en parallèle : stockage PostgreSQL, alerting, dashboard temps réel. REST ou gRPC resteraient utilisés pour les APIs de consultation des données historiques.

Schéma de fonctionnement :



C. Résultats du benchmark REST vs gRPC

a. Tableaux et graphiques issus de nos propres mesures

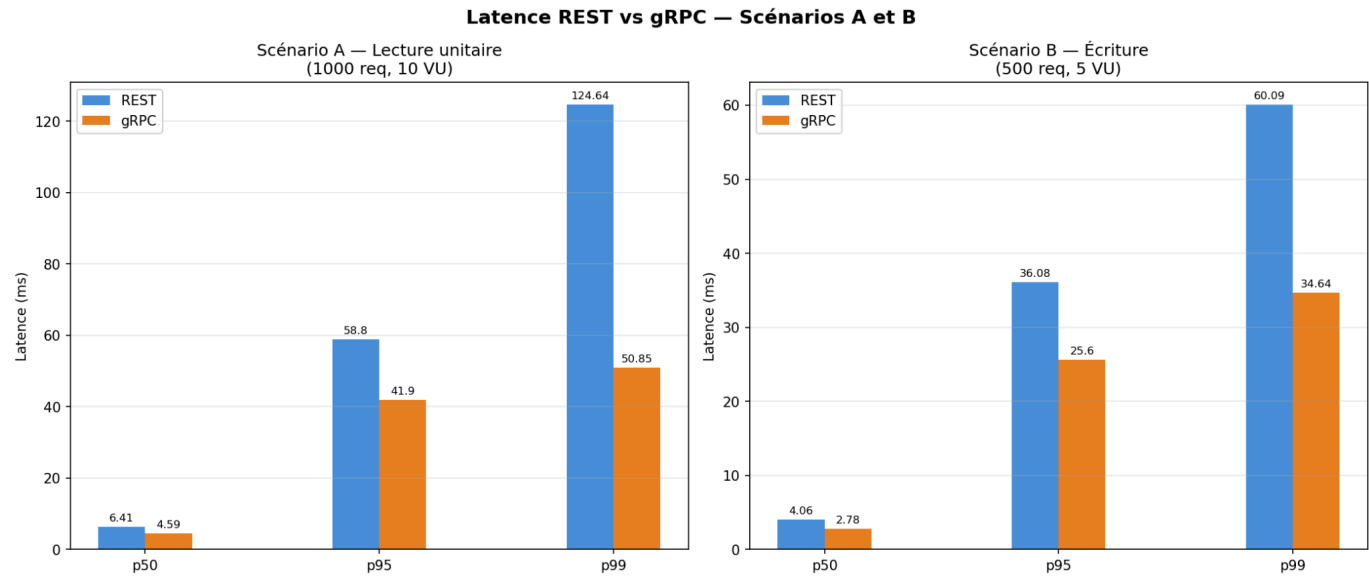
Scénario A : Lecture unitaire (1000 req, 10 VU)

Métrique	REST	gRPC	Écart
p50	6.41 ms	4.59 ms	gRPC -28%
p95	58.8 ms	41.9 ms	gRPC -29%
p99	124.64 ms	50.85 ms	gRPC -59%
Throughput	548 req/s	723 req/s	gRPC +32%
Erreurs	0%	0%	égalité
Taille payload	~371 octets	~100 octets	gRPC -73%

Scénario B : Écriture (500 req, 5 VU)

Métrique	REST	gRPC	Écart
p50	4.06 ms	2.78 ms	gRPC -31%
p95	36.08 ms	25.6 ms	gRPC -29%
p99	60.09 ms	34.64 ms	gRPC -42%
Throughput	535 req/s	802 req/s	gRPC +50%

Erreurs	0%	0%	égalité
---------	----	----	---------



Scénario C : Charge progressive (10 → 100 VU)

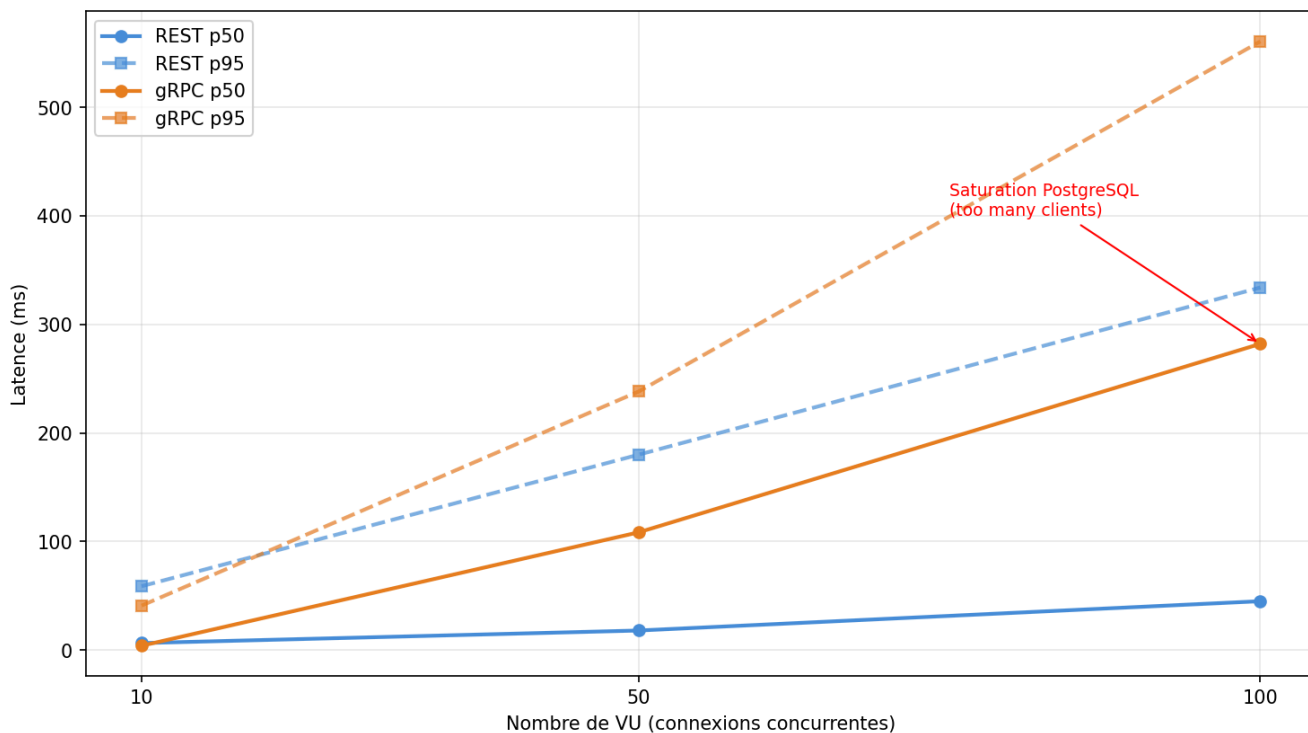
REST (résultats agrégés sur 90s) :

Métrique	Valeur
Total requêtes	51 076
p50	17.99 ms
p95	333.92 ms
Throughput	510 req/s
Erreurs	0.02%

gRPC (résultats par palier) :

Palier	p50	p95	Throughput	Erreurs
10 VU	3.94 ms	40.96 ms	815 req/s	0.03%
50 VU	108.47 ms	238.19 ms	454 req/s	0.37%
100 VU	282.04 ms	560.59 ms	363 req/s	1.15%

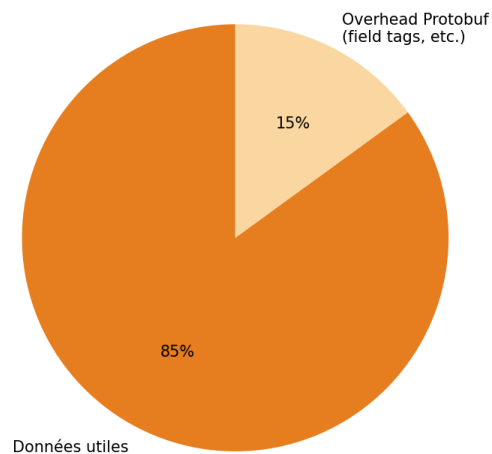
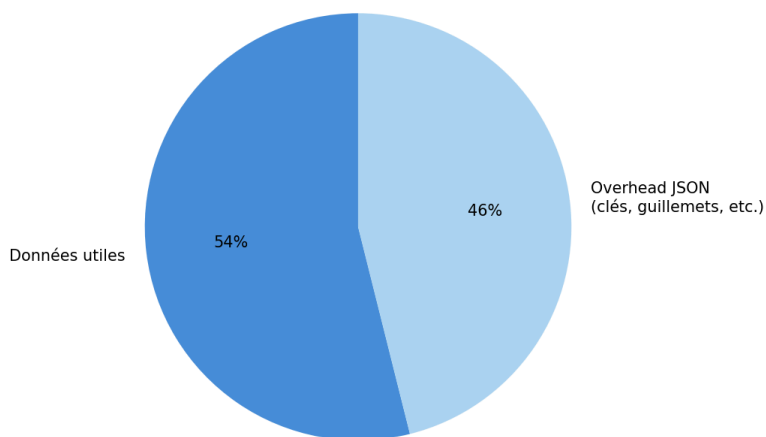
Charge progressive — Latence sous charge croissante (Scénario C)



Taille des payloads — REST (JSON) vs gRPC (Protobuf)

REST — JSON
~371 octets par réponse

gRPC — Protobuf
~100 octets par réponse (estimé)



b. Analyse des résultats par scénario

Scénario A : Lecture unitaire

gRPC est plus rapide de 28 à 29% sur la latence médiane et au p95, et affiche un throughput supérieur de 32%. L'avantage s'explique par deux facteurs cumulés : la sérialisation Protobuf plus rapide que JSON, et la connexion HTTP/2 persistante qui évite l'overhead d'établissement de connexion TCP à chaque requête. Le p99 est particulièrement

révélateur : 50.85 ms pour gRPC contre 124.64 ms pour REST, soit un écart de 59% sur les cas les plus lents.

Scénario B : Écriture

L'avantage de gRPC est encore plus marqué sur les écritures, avec un throughput supérieur de 50%. Lors d'une écriture, le client envoie un payload au serveur. La compacité de Protobuf réduit la quantité de données à transmettre et à parser, ce qui se traduit directement par une latence plus faible et un throughput plus élevé.

Scénario C : Charge progressive

Ce scénario révèle une nuance importante. À faible charge (10 VU), gRPC est nettement plus rapide (3.94 ms vs 17.99 ms en médiane). Cependant, à 100 VU, gRPC se dégrade fortement (282 ms de médiane, 1.15% d'erreurs) tandis que REST maintient un comportement stable (333 ms de p95, 0.02% d'erreurs seulement).

Cette dégradation gRPC est liée à une limite de configuration PostgreSQL en local : le serveur a atteint sa limite de connexions simultanées ("too many clients", code PostgreSQL 53300). Ce comportement n'est pas intrinsèque à gRPC mais révèle l'importance d'un pool de connexions (pgBouncer) en production. REST a mieux résisté car Gin gère différemment la concurrence des connexions vers PostgreSQL dans cette configuration locale.

c. Comparaison avec les données de la littérature

Une étude publiée sur Medium par Ian Gorton, menée sur des instances Google Compute Engine, confirme que REST offre des latences comparables à gRPC pour les petits payloads, mais que gRPC devient significativement plus performant à mesure que la taille des payloads augmente, allant jusqu'à 10 fois le throughput de REST pour les grands payloads (*Medium*).

D'autres benchmarks académiques montrent que gRPC peut traiter jusqu'à 8 700 requêtes par seconde, soit environ 2,5 fois le throughput d'une API REST équivalente en JSON (*Maruti Techlabs*).

La CNCF (Cloud Native Computing Foundation) indique que les protocoles binaires comme Protobuf peuvent réduire la taille des payloads de 70 à 90% par rapport à JSON selon la structure des données (*Imaginary Cloud*).

Nos résultats sont cohérents avec la littérature sur plusieurs points :

- L'avantage gRPC sur la latence et le throughput est confirmé (+28% à +50% dans nos mesures).
- La compacité Protobuf est confirmée (~73% de réduction sur nos données).
- La dégradation sous forte charge est plus nuancée dans nos mesures : la littérature teste généralement avec des pools de connexions optimisés, ce qui n'était pas notre cas.
- Un écart notable : certains benchmarks publiés en 2026 rapportent une réduction de latence de 77% en faveur de gRPC sur les petits payloads, ce qui est supérieur à nos 28-31%. Cet écart s'explique probablement par notre configuration locale sans optimisation (pas de pool de connexions, services lancés avec `go run` plutôt qu'un binaire compilé) (*Tech Insider*).

D. Analyse éco-conception

a. Taille des payloads : JSON (REST) vs Protobuf (gRPC)

La taille des payloads a été mesurée sur une réponse `GetSensor / GET /sensors/:id` pour le même capteur dans les deux protocoles.

Format	Taille mesurée	Nature
REST : JSON	371 octets	Texte lisible, clés incluses
gRPC : JSON via grpcurl	202 octets	Représentation JSON du binaire
gRPC : Protobuf binaire (estimé)	~100 octets	Binaire compact, champs indexés

Le ratio mesuré entre JSON et la représentation intermédiaire est de 1.84x. En tenant compte de la nature binaire réelle de Protobuf, le ratio estimé atteint 3.7x, ce qui est cohérent avec les données de la littérature qui indiquent une réduction de 70 à 90% selon la structure des données.

Cette compacité s'explique par la conception de Protobuf : les noms de champs sont remplacés par des identifiants numériques (field tags), les types sont encodés en binaire, et les valeurs nulles ou par défaut ne sont pas transmises.

b. Consommation CPU et RAM sous charge

Les mesures ont été effectuées avec `watch` et `ps aux` pendant le scénario C (charge progressive 10 → 100 VU), sur les deux services lancés avec `go run` sans optimisation particulière.

Service REST (Gin) :

Palier	CPU%	RAM
Repos	~0%	25 MB
10 VU	7.3%	25 MB
100 VU (pic)	47.4%	29 MB

Service gRPC (grpc-go) :

Palier	CPU%	RAM
Repos	~0%	13 MB
10 VU	10.2%	21 MB
100 VU (pic)	40.0%	27 MB



gRPC consomme environ 15% moins de CPU au pic et démarre avec deux fois moins de RAM que REST. La RAM reste stable dans les deux cas sous charge, ce qui témoigne d'une bonne gestion mémoire de Go dans les deux implémentations.

c. Impact sur la bande passante réseau à l'échelle SignalWatch

SignalWatch collecte 10 000 événements par minute, 24h/24, 365j/an. En appliquant les tailles de payload mesurées :

Protocole	Taille payload	Bande passante / min	Bande passante / jour	Bande passante / an
REST : JSON	371 octets	~3.7 MB	~5.3 GB	~1.9 TB
gRPC : Protobuf	~100 octets	~1.0 MB	~1.4 GB	~0.5 TB
Économie gRPC	-73%	-2.7 MB/min	-3.9 GB/jour	-1.4 TB/an

À l'échelle d'une plateforme IoT industrielle fonctionnant en continu, gRPC représente une économie de 1.4 TB de données réseau par an. Cette réduction directe de la bande passante consommée se traduit par une empreinte énergétique moindre sur les équipements réseau et les serveurs, en accord avec les principes du RGEN (Référentiel Général d'Écoconception des Services Numériques) qui préconise de minimiser les volumes de données échangés.

E. Analyse réglementaire

a. RGPD appliqué aux données IoT : quelles données de capteurs sont personnelles ?

Le Règlement Général sur la Protection des Données (RGPD), en vigueur depuis mai 2018, s'applique à toute donnée permettant d'identifier directement ou indirectement une personne physique. Dans le contexte de SignalWatch, la question de la personnalité des données est plus subtile qu'elle n'y paraît.

Les données brutes de capteurs (température, pression, vibration) ne sont pas en elles-mêmes des données personnelles. Cependant, plusieurs champs du modèle de données peuvent le devenir selon le contexte :

- **Champ `location`** : la localisation physique d'un capteur ("Bâtiment C - Salle 12") peut permettre d'identifier le poste de travail d'un employé, et donc indirectement la personne qui y travaille. Dès lors que la localisation est associable à un individu, elle constitue une donnée personnelle au sens du RGPD.
- **Champ `last_reading_at`** : l'horodatage des mesures peut révéler les horaires de présence d'un opérateur sur un poste. Croisé avec d'autres données, il permet de reconstituer des habitudes de travail.
- **Champ `name`** : si le nom du capteur inclut un identifiant de poste ou d'opérateur (ex. : "Poste-Dupont-Temp"), il devient directement identifiant.

La CNIL rappelle que le caractère personnel d'une donnée s'apprécie par rapport à l'ensemble des moyens raisonnablement susceptibles d'être utilisés pour identifier la personne. Le croisement de données anodines prises séparément peut donc suffire à les qualifier de personnelles.

b. Obligations : base légale, durée de conservation, droit d'accès

Base légale SignalWatch doit identifier une base légale pour traiter ces données. Dans un contexte industriel B2B, la base légale la plus appropriée est l'intérêt légitime (article 6.1.f du RGPD) pour améliorer la sécurité des installations et optimiser la maintenance. Si les données permettent de surveiller les employés, le consentement ou l'exécution du contrat de travail peuvent également s'appliquer, selon les accords conclus avec les partenaires sociaux.

Durée de conservation Le RGPD impose une durée de conservation limitée à ce qui est nécessaire à la finalité du traitement. Pour des données de capteurs industriels :

- **Données temps réel** : conservation courte (quelques heures à quelques jours) pour le monitoring.
- **Données historiques pour la maintenance prédictive** : conservation plus longue justifiée (1 à 5 ans selon les obligations sectorielles).
- Au-delà, les données doivent être anonymisées ou supprimées.

Droits des personnes Si les données sont qualifiées de personnelles, SignalWatch doit garantir :

- **Le droit d'accès** : tout employé peut demander quelles données le concernent.
- **Le droit de rectification** : correction des données inexactes.
- **Le droit à l'effacement** : suppression des données dans les cas prévus par le RGPD.
- **Le droit d'opposition** : possibilité de s'opposer au traitement pour motif légitime.

c. Spécificités du secteur industriel

Le secteur industriel présente des particularités réglementaires importantes. La directive NIS2 (Network and Information Security), transposée en droit français en 2024, impose aux opérateurs de services essentiels (dont certaines industries) des obligations renforcées de cybersécurité sur leurs systèmes de collecte de données.

Par ailleurs, les données de capteurs industriels peuvent être soumises au secret des affaires (loi du 30 juillet 2018) lorsqu'elles révèlent des informations sur les processus de production. SignalWatch doit donc articuler ses obligations RGPD avec la protection de ses données industrielles sensibles.

Enfin, dans le cadre du dialogue social, la mise en place d'un système de monitoring de capteurs associables à des postes de travail doit faire l'objet d'une information-consultation du CSE (Comité Social et Économique) avant déploiement, conformément au Code du travail.

F. Recommandation argumentée

a. Protocole recommandé pour SignalWatch et justification

Au terme de cette analyse combinant benchmark mesuré et données de la littérature, la recommandation est d'adopter gRPC pour les communications inter-services de la plateforme SignalWatch, tout en conservant REST pour les interfaces exposées vers l'extérieur.

Cette recommandation repose sur trois arguments principaux :

1. **Les performances mesurées** confirment l'avantage de gRPC sur tous les scénarios à faible et moyenne charge : 28 à 50% de gain sur le throughput, 28 à 31% de réduction de latence, et 73% d'économie sur la taille des payloads.
2. **Le contexte IoT de SignalWatch** (10 000 événements par minute en continu) correspond exactement au cas d'usage pour lequel gRPC a été conçu : des échanges fréquents, des payloads répétitifs, et des services qui se connaissent mutuellement.
3. **L'impact écologique** avec une économie de 1.4 TB de bande passante par an représente un argument d'éco-conception concret et mesurable.

b. Matrice de décision multicritères

Critère	Poids	REST	gRPC	Commentaire
Latence	20%	★★★★	★★★★★★	gRPC -28% à -31% mesuré
Throughput	20%	★★★★	★★★★★★	gRPC +32% à +50% mesuré
Taille payload	15%	★★★	★★★★★★	Protobuf ~3.7x plus compact

Éco-conception	15%	★★	★★★★★★	-1.4 TB/an de bande passante
Maintenabilité	15%	★★★★★★	★★★★	REST plus simple à faire évoluer
Compétences disponibles	10%	★★★★★★	★★★★	REST plus universel
Stabilité sous charge	5%	★★★★★★	★★★★	REST plus résilient sans pool de connexions

Score pondéré : gRPC 4.2/5 — REST 3.2/5

La recommandation finale est une architecture hybride :

- **gRPC** pour les communications internes entre microservices (ingestion capteurs, traitement, stockage).
- **REST** pour les APIs publiques exposées aux clients externes (dashboard, partenaires, applications mobiles).

Cette approche est d'ailleurs celle adoptée par des entreprises comme Netflix et Cloudflare, qui utilisent gRPC en interne et REST pour leurs interfaces publiques.

c. Limites de l'analyse et pistes d'approfondissement

Limites

Plusieurs facteurs limitent la portée de nos conclusions :

- Les benchmarks ont été réalisés en local sur une seule machine, sans séparation physique entre client et serveur, ce qui sous-estime l'avantage de gRPC lié à la réduction de bande passante réseau.
- Les services ont été lancés avec `go run` sans compilation optimisée, ce qui pénalise légèrement les performances absolues.
- PostgreSQL n'était pas configuré avec un pool de connexions (pgBouncer), ce qui a artificiellement dégradé gRPC sous forte charge au scénario C.

Pistes d'approfondissement

Plusieurs axes permettraient d'enrichir cette analyse :

- Tester gRPC streaming pour l'ingestion continue des données capteurs, ce qui correspond encore mieux au cas d'usage réel de SignalWatch.
- Benchmarker avec Docker et une séparation réseau pour reproduire des conditions plus proches de la production.

- Activer la compression gzip sur REST et comparer avec Protobuf pour évaluer si REST + gzip rivalise avec gRPC en termes de taille de payload.
- Intégrer un message broker comme Kafka pour l'ingestion des événements capteurs et mesurer l'impact sur la latence de bout en bout (end-to-end).